Muhammad Adil Saeed.
24k-0715.
COAL - Assignment 3.
BCS-3J.

Question 1:

```
Include irvine32.inc.
.data.
        dividend dword 0D4A4h.
        divisor dword 0Ah.
        msg BYTE "Function Ended",0.
        emsg. BYTE "EAX = ",0

.code.
main proc.
        mov eax, dividend.
        mov ebx, divisor
        call dividenUntil

exit
main endp.

divideUntill PROC.
    xor edx, edx.
    div ebx
    mov edx, offset emsg.
    call writestring
    call writeeax
    call crlf.
    cmp eax, 5h.
    je L1
    call divideUntill
    ret.

L1:
    mov edx, offset msg.
    call writeString
    ret

divideUntill Endp.

end main.
```

Question 2:

```
Include irvine32.inc .
.data.
 userInput byte 30 DUP(?)
 msg Byte "Enter the String: ",0 .
 countofA   dword 0
 countofE.  dword 0 .
 countofI   dword 0 .
 countofO   dword 0 .
 countofU   dword 0 .
 StringLength dword 0?
 messageA Byte "A or a = ",0 .
 messageE byte "E or e = ",0
 messageI byte "I or i = ",0 .
 messageO byte "O or o = ",0 .
 messageU byte "U or u = ",0.

.code
 main proc.
      mov edx, offset msg .
      call writeString
      mov edx, offset userInput
      mov ecx, 30
      call ReadString
      mov esi, offset userInput.
      xor ecx, ecx .

 calcLength:
      cmp byte ptr[esi], 0 .
      je LengthCovered .
      inc ecx .
      inc esi
      jmp calcLength .

 LengthCovered .
      mov stringLength, ecx .
      xor esi, esi .

 findingVowels:
      cmp esi, string length
      je doneCounting
      mov al, userInput[esi] .
```

- - - - - - -

2 -

```
cmp al, 'a'
je incA
cmp al, 'A'
je incA.
cmp al, 'E'
je incE
cmp al, 'e'
je incE
cmp al, 'I'
je incI
cmp al, 'I'
je incI
cmp al, 'o'
je incO
cmp al, 'O'
je incO.
cmp al, 'u'
je incU
cmp al, 'U'
je incU.

nextChar:
    inc esi
    jmp findVowels.
incA:
    inc countA
    jmp nextChar.
incE:
    inc countE
    jmp nextChar.
incI:
    inc countI
    jmp nextChar.
incO:
    inc countO.
    jmp nextChar.
incU:
    inc countU
    jmp nextchar.

doneCounting:
    mov edx, offset messageA.
    call writestring
    mov eax, countA.
    call writeDec.
    call crlf.

    move edx, offset messageE
    call writestring
    mov eax, countE
    call writeDec.
    call crlf.

    mov edx, offset messageI
    call writestring
    mov eax, countI
    call writeDec.
    call crlf.

    mov edx, offset messageO.
    call writestring
    mov eax, countO
    call writeDec.
    call crlf.

    mov edx, offset messageU.
    call writestring
    mov eax, countU.
    call writeDec.
    call crlf.

    exit.
main endp.
end main.
```

Question 3:

```
Include irvine32.inc
.data
  str1 byte "###FAST",0.
  trimChar byte "#"

.code
  main proc
      mov edi, offset str1.
      mov al, trimChar.
      call StrTrim

      mov edx, offset str1
      Call writestring.

      exit.
  main endp.

  StrTrim Proc
      push esi
      push ecx.

      mov esi, edi

  findFirst:
      cmp byte ptr[esi], 0.
      je done

      cmp byte ptr[esi], al
      jne done
      inc esi
      jmp findFirst

  done:
      cmp esi, edi
      je finish.

      mov ecx, 0
      mov edx, esi
      Count Remaining:
          cmp byte ptr[edx], 0
          je copyChars.
          inc ecx.
          inc edx
          jmp count Remaining
      copyChars:
          inc ecx.
          mov esi, esi
          mov edi, edi
          rep movsb.

      finish:
          pop ecx.
          pop esi
          ret
  StrTrim ENDP
  end main.
```

4.

```
FindFive Proc. arrayPtr: DWORD, arraySize: Dword

        push esi
        push edi
        push ecx.
        push edx.

        mov esi, arrayPtr.
        mov ecx, arraySize.
        cmp ecx, 5.
        jl notfound.
        xor edi, edi

checkLoop:
        mov eax, ecx.
        sub eax, 4
        cmp edi, eax.
        jge notfound.
        xor edx, edx.

checkfive:
        cmp edx, 5
        je found
        mov eax, edi
        add eax, edx.
        shl eax, 2
        cmp dword ptr [esi+eax], 5.
        jne nextIndex.
        inc edx.
        jmp checkfive.

nextIndex:
        inc edi
        jmp checkLoop.

found:
        mov eax, 1
        jmp exitProc.

notfound:
        xor eax, eax.

*continued.
```

```
*
    exitProc:
        pop edx.
        pop ecx.
        pop edi
        pop esi
        ret 8.
    FindFive endp.

end main.
```